PRACTICAL 13

Aim :

Integrate an external AI service into the Task Management application securely. Generate an editable task description from a task title using an Express backend route, keep the API key server-side, handle failures gracefully, and prevent repeated requests with a basic rate limit.

Code :

backend/routes/ai.js :-

```
router.post('/generate-description', asyncHandler(async (req, res) => {
  const title = typeof req.body.title === 'string'
    ? req.body.title.trim() : '';
  if (title.length < 2) {
    return res.status(400).json({ error: 'Enter a task title first' });
  }
  if (!process.env.OPENAI_API_KEY) {
    return res.status(200).json({
      description: '',
      fallback: true,
      message: 'AI suggestions are unavailable. Write manually.'
    });
  }
  // Call the provider from the backend only.
  const response = await fetch('https://api.openai.com/v1/chat/completions', {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${process.env.OPENAI_API_KEY}`,
      'Content-Type': 'application/json'
    },
```

```
    body: JSON.stringify({
      model: 'gpt-4o-mini',
      messages: [{ role: 'user', content: `Write a short description for: ${title}` }]
    })
  });
  if (!response.ok) throw new Error('AI provider request failed');
  const result = await response.json();
  res.json({ description: result.choices[0].message.content, fallback: false });
}));
```

Code :

frontend/src/api.js :-

```
export const generateDescription = (title) => request('/ai/generate-description', {
  method: 'POST',
  body: JSON.stringify({ title })
});
```

frontend/src/pages/Projects.jsx :-

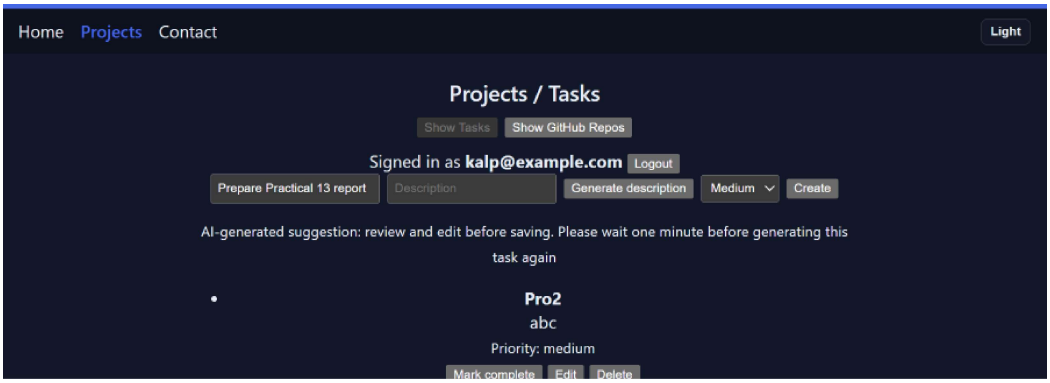
```
function handleGenerateDescription() {
  setAiLoading(true);
  setAiMessage(null);
  generateDescription(title)
    .then((result) => {
      if (result.description) setDescription(result.description);
      setAiMessage(result.message || 'AI suggestion added. Review it before saving.');
    })
    .catch((err) => setAiMessage(err.message))
    .finally(() => setAiLoading(false));
}
```

backend/.env :-

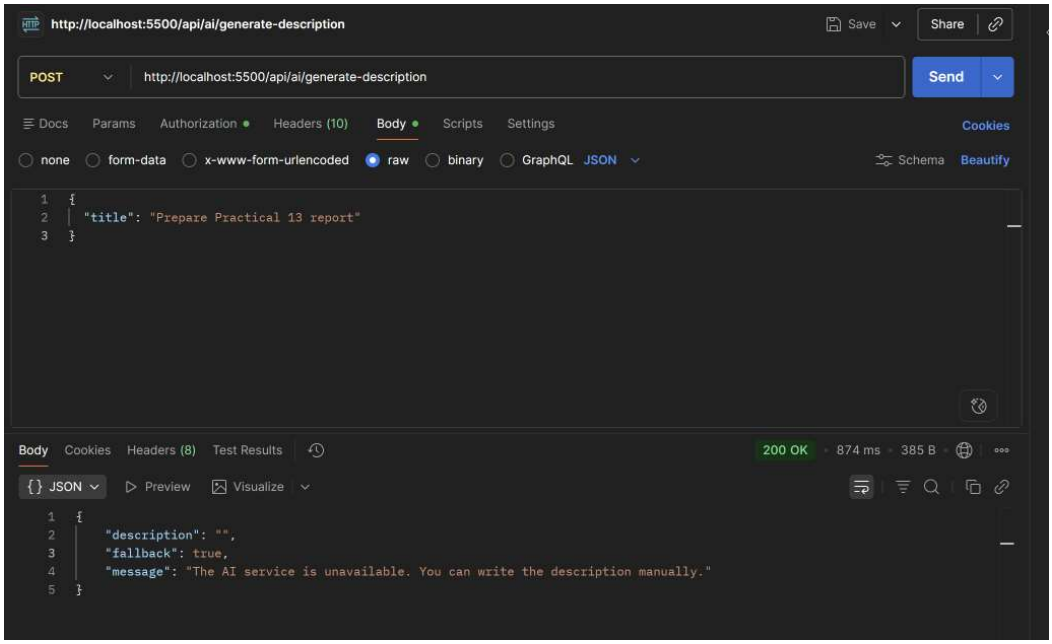
```
OPENAI_API_KEY=your_private_key_here
```

The real key is stored only in the backend environment and is not committed or exposed to the frontend.

Output :







Result :

The AI-powered task description feature was integrated through the backend. The API key remains server-side, the generated description is editable in the React form, failures return a friendly fallback, and repeated requests are limited per task.